

Trading Journal

AI Agent Architecture

Multi-Model Intelligence Framework — v1.0.1

Claude Sonnet · Claude Haiku · Google Gemini · xAI Grok · Nansen · CoinGecko

What this document covers

This document describes every AI agent, data agent, and automation agent that powers the self-hosted crypto futures trading journal. Each section explains what the agent does, why it was designed that way, what model it uses, when it fires, and how it connects to the others. Suitable for beginners and experts.

1. System Overview

The trading journal is a self-hosted application running on a Raspberry Pi 5. It connects to cryptocurrency futures exchanges (Bitget, Blofin), tracks your trade history, and uses a network of specialised AI agents to help you make better trading decisions. Every agent has a specific, focused responsibility — none of them tries to do everything.

For **beginners**: think of it as a team of analysts, each an expert in one area. One analyst reads the charts, another checks social media, a third scores your trade ideas, another reviews your history. The orchestrator is the team leader who decides who speaks and how much weight to give each opinion.

For **experts**: the architecture uses a prompt-caching-aware stable/dynamic context split, a consensus scoring layer across two independent LLMs with divergence detection, MC-weighted external intelligence (Grok), and a backtest feedback loop that injects historical win-rate patterns directly into every scoring prompt.

Three types of agents

Type	What it means	Examples
Analysis Agents	Receive input (a trade call, a position, historical data) and analyze, advise, or recommend trade assessments	data_analyzer, advisor, trend_spotter, trade_assessment
Automation Agents	Run on a schedule in the background without any user action. They fetch data, detect setups, send a	scan_scheduler, data_fetcher, detect_setups, send_a
Data Agents	Fetch and cache external data (no AI). They are called by analysis agents to enrich prompts with e	crypto_prices, crypto_news, crypto_sentiment, market_a

2. Master Orchestrator

The orchestrator (`agent_orchestrator.py`) does not call any AI model itself. Its job is to coordinate results from multiple models and make routing decisions. It answers two questions: *which model should handle this task?* and *do Claude and Gemini agree on this trade signal?*

2.1 Model Router

Every AI task in the journal is classified as either a **reasoning task** (needs the most capable model) or a **classification task** (can be done by a faster, cheaper model). This matters because Claude Sonnet costs roughly 10× more per token than Claude Haiku — routing correctly reduces costs without sacrificing accuracy.

Task	Model	Reason for choice
call_analyzer	Sonnet 4.6	Complex structured JSON output with 15+ fields, chain-of-thought scoring
scanner_batch	Sonnet 4.6	Evaluates 12 symbols simultaneously, needs nuanced entry/SL/TP reasoning
advisor	Sonnet 4.6	Full portfolio coaching from 800+ trades, long-form recommendations
rulebook	Sonnet 4.6	Synthesises entire trade history into personalised trading rules
limit_analyzer	Sonnet 4.6	Risk decision on a pending order — accuracy critical
pattern_detector	Sonnet 4.6	Cross-pattern compounding analysis — needs genuine reasoning
scanner_quick	Haiku 4.5	Score 0-10 + one sentence — pure classification, runs 100× per scan
live_trade	Haiku 4.5	Hold/Close/Adjust action — simple rubric, latency matters
hindsight	Haiku 4.5	Retroactive ENTER/SKIP verdict — binary classification task
trade_grader	Haiku 4.5	A/B/C/D execution grade — simple rubric, runs once per closed trade

2.2 Consensus Scoring Algorithm

When a user analyzes a trade call, both Claude and Google Gemini score the setup independently. Claude receives the full context (rulebook, chart data, market conditions, similar historical trades). Gemini receives *only the raw call text* — no extra context. This is intentional: two assessors with different information sets produce more meaningful agreement or disagreement than two copies of the same prompt.

When they agree ($|\Delta| \leq 1$), confidence is high — the setup signal is robust. When they strongly disagree ($|\Delta| > 3$), the trade is flagged for manual review before acting on it. The weighted average (Claude 60%, Gemini 40% on mild disagreements) reflects that Claude's full context generally produces more accurate scores for structured technical setups.

Delta ($ \text{Claude} - \text{Gemini} $)	Confidence	Score used	UI flag	Recommended action
0 – 1 point	High	Simple average	✓ Confirmed	Trade with normal risk
2 points	Medium	Simple average	~ Aligned	Trade, monitor closely
3 points	Low	Claude 60% + Gemini 40%	⚠ Divergent	Reduce size or wait for confirmation
> 3 points	Very Low	Claude score kept	■ REVIEW	Do not trade — investigate the disagreement

3. Analysis Agents

Analysis agents are triggered by user actions (clicking a button, requesting an analysis). Each one receives a focused input, builds an optimised prompt from the shared context system, calls the AI, and returns structured JSON.

■ Call Analyzer — ai_call.py

Sonnet 4.6

Trigger: User pastes analyst call text (Telegram, Twitter, manual)

Output: Score 1-10, entry/SL/TP, sizing, pattern warnings, R:R, Gemini consensus

The primary intelligence agent for evaluating trade calls from external analysts.

When a user receives a trade call from an analyst on Telegram, they paste it here. The agent automatically extracts the symbol, direction, entry price, stop loss and take profit levels using regex. It then runs three things in parallel: (1) an ATR-based stop loss quality check using 1H candle data, (2) live market context (funding rates, Fear & Greed), and (3) Gemini's independent pre-proof score. Claude then receives the full context — personalised rulebook, calibration feedback, chart indicators for 4H and 1D, Nansen smart money signal, Grok social sentiment (weighted by market cap), similar historical trades — and produces a 15-field structured analysis. The previous analysis for the same symbol is injected as a learning loop (CoT reuse), enabling Claude to detect if setup conditions have changed. The consensus score (Claude vs Gemini) is saved to the database alongside the full analysis.

■ Scanner — ai_scanner.py (3-stage)

Sonnet + Haiku

Trigger: Every 30 minutes (automatic) or manual run

Output: Up to 12 scored setups with entry_zone, sl, tp1, tp2, rr_ratio, urgency

Proactively finds trade setups across 100 symbols without waiting for analyst calls.

Stage 1 (Confluence Filter, no AI): fetches 4H and 1D candles for all 100 symbols in parallel and computes RSI, MACD, EMA, ADX, WaveTrend, and CVD. Symbols where fewer than 2 indicators agree on direction are eliminated. This typically cuts 100 to ~25-30 candidates with zero API cost. Stage 2 (Quality Gate, no AI): applies technical rules — rejects overextended RSI, missing S/R structure, flat ADX (choppy market), very high funding rate. This cuts to ~10-15 finalists. Stage 3a (Haiku Quick Score): Haiku scores each finalist with a minimal prompt (120 tokens output max) — faster and 10× cheaper than Sonnet. Setups scoring below threshold are dropped. Stage 3b (Sonnet Batch): all remaining finalists are scored in a SINGLE Sonnet call using a batch prompt. This is a key token optimisation — scoring 12 symbols simultaneously rather than 12 sequential calls. Stage 3c (Gemini Consensus): top-5 finalists receive an independent Gemini score. The final ranking adjusts based on consensus confidence. Alerted setups are automatically saved to analyzed_calls so they can be linked to live positions without manual intervention.

■ AI Advisor — ai_advisor.py

Sonnet 4.6

Trigger: User clicks 'Get AI Advice' in the Edge Lab

Output: Portfolio strengths, weaknesses, specific recommendations, symbol insights

High-level portfolio coaching based on full trade history and current market.

The advisor receives aggregated statistics from your entire trade history: win rate, profit factor, performance by symbol, by weekday, by hour, by setup type, by duration. It also receives the current market context (BTC dominance, Fear & Greed, funding rates). The rulebook and calibration data are cached as the stable prefix. Claude identifies patterns across all dimensions and produces a structured coaching report: what you're doing well (strengths), where you're losing money (weaknesses), and 3-5 specific actionable recommendations with data to back them up. For example: 'Your Friday afternoon LONG trades have a 43% win rate vs 71% on other days — consider avoiding new positions after 15:00 UTC on Fridays.'

■ Hindsight Analyzer — ai_hindsight.py

Haiku 4.5

Trigger: User runs hindsight batch (last N trades)

Output: ENTER/SKIP verdict per trade, TP/FP/TN/FN accuracy, comparison P&L;

Retroactive blind scoring: what would AI have said if it saw the setup before the outcome?

This is a backtesting tool with a key discipline: Claude is shown the technical picture as it appeared AT ENTRY TIME — not the current chart. The agent fetches historical OHLCV candles ending at the exact trade entry timestamp and reconstructs the indicator state. It then scores the setup without knowing how the trade ended. The results show: (1) how often Claude's ENTER calls actually won (True Positive rate), (2) how often its SKIP calls would have been correct, and (3) the hypothetical P&L; if you had only taken trades Claude would rate ≥ 6 . This calibration loop helps identify whether the AI's scoring is predictive of actual outcomes — the foundation for the 85% accuracy target.

■ Live Trade Checker — ai_live_trade.py

Haiku 4.5

Trigger: User clicks '■ AI Analysis' on a live position card

Output: Action (Hold/Close/Adjust SL), risk rating 1-10, TP/SL suggestions

Per-position quick health check for open futures positions.

Receives the full position data (entry, mark, SL, TP, duration, margin, unrealized P&L, funding rate) plus the current 4H chart indicators. Haiku evaluates whether the trade thesis is still valid, whether the stop is at risk, and whether the position has been open too long. Returns a structured recommendation in under 2 seconds. Haiku is used here specifically because the user is looking at their live portfolio and latency matters — a 5-second wait for each card would be frustrating.

■ Limit Analyzer — ai_limit.py

Sonnet 4.6

Trigger: User clicks 'Analyze' on a pending limit order

Output: Entry quality score, risk assessment, ATR validation

Evaluates limit orders before they trigger — catching bad setups before they fill.

Limit orders represent planned trades that haven't executed yet. The limit analyzer checks whether the planned entry price is at a structurally sound level, whether the stop loss is outside the ATR noise floor, and how this limit fits with other open or pending positions. Uses Sonnet because this is a consequential decision (real money at risk when the limit fills) and the nuanced entry quality assessment benefits from the more capable model.

■ Trade Grader — ai_trade_grader.py

Haiku 4.5

Trigger: User clicks '■ Grade' on any closed trade

Output: Grade A/B/C/D with written explanation of execution quality

Execution quality feedback loop — was the entry/exit actually well-executed?

Separate from whether the trade was profitable, the grader evaluates *how well* the trade was executed: did the entry happen near the ideal zone, was the stop set correctly relative to structure, was the exit too early or too late, was risk sizing appropriate? A trade can be a B+ execution that still lost money (correct process, bad luck) or a D execution that won by chance. Tracking execution grades over time identifies whether losses come from bad setups or bad execution — very different problems requiring different solutions.

■ Rulebook Generator — ai_rulebook.py

Sonnet 4.6

Trigger: Weekly auto-regen (if 5+ new trades) or manual update

Output: 10 personalised rules with confidence levels and stale annotations

Self-updating trading rulebook synthesised from your actual trade history.

The rulebook is the most impactful agent for long-term accuracy improvement. Claude reads your complete trade statistics — performance by setup type, symbol, session, weekday, hour, holding period, and direction — and synthesises 5-10 personalised rules. These are not generic advice but rules derived from YOUR specific data. For example: 'Friday morning breakouts: 3 trades, 0 wins, avg -\$166 — strong evidence to avoid.' Rules older than 30 days are annotated [stale] so Claude discounts them in analysis prompts. A regen guard prevents regeneration if fewer than 5 new trades exist since the last update — avoiding rules based on too little data. All 10 rules are injected into every analysis prompt as the cached stable prefix, meaning Claude always has your personalised context without paying for it on every call.

4. External AI Providers

Two external AI providers are integrated alongside Claude to provide independent signals that cannot come from technical analysis alone.

4.1 Google Gemini — Independent Pre-Proof Scorer

Why Gemini? Having a second AI model score the same trade call independently creates a cross-validation signal. Gemini sees only the raw call text — no rulebook, no chart context. This information asymmetry is deliberate. If two models with different training data and different context reach the same score, the signal is stronger. If they strongly disagree, that disagreement is itself informative.

For beginners: Imagine having two doctors give independent opinions on the same X-ray without telling either what the other said. If both say 'this looks fine,' you're confident. If one says 'this is serious' and the other says 'nothing to worry about,' you know to get a third opinion.

Technical details: Uses Gemini 2.0 Flash (fast, cheap) via the Google Generative Language API with `responseMimeType: application/json` to force structured output. Runs in parallel with ATR checks and market context fetches — no additional wall-clock time. Cached 30 minutes per (symbol, direction) pair. Results stored in `analyzed_calls.gemini_score` and `consensus_score` columns for backtesting.

4.2 xAI Grok — Social Intelligence (X/Twitter)

Why Grok? Grok has real-time access to X (Twitter) data. For small-cap and micro-cap crypto assets, the price is often driven more by social narrative and community sentiment than by on-chain fundamentals or technical patterns. Grok is the only AI in the stack that can see what people are saying about a specific coin right now.

Market cap weighting: For large-cap coins like Bitcoin or Ethereum, social media noise far exceeds the signal — institutional trading dominates, and a viral tweet rarely moves the price meaningfully. For micro-cap coins (\$200M market cap or below), a single influential post can move price 30%. The weight formula reflects this reality:

Market Cap	Grok Weight	Rationale
> \$5 billion	0% — skipped	Large cap: social noise > signal. Institutional flows dominate.
\$1B – \$5B	15% weight	Mid-large: supplementary context only.
\$200M – \$1B	40% weight	Small cap: social sentiment is a meaningful price driver.
< \$200M	80% weight	Micro cap: social narrative is often the PRIMARY driver.
Unknown market cap	60% weight	Treated as small-cap until CoinGecko confirms otherwise.

What Grok provides: a 100-130 word brief covering X/Twitter sentiment (bullish/bearish/mixed and dominant narratives), recent news or developments (last 7 days), social quality assessment (organic analysis vs. coordinated hype), and the biggest social/news risk to the current trade direction. Red flags are explicitly marked with ■. The brief is injected into the prompt context with a label showing the weight so Claude knows how much to rely on it relative to technical indicators.

5. Data Agents

Data agents fetch, cache, and format external data. They don't call any AI model — they are pure data pipelines whose output enriches AI prompts.

Agent	Source	Cache	What it provides	Injected into
chart_context.py	Bitget REST v2	10 min	OHLCV candles → RSI, MACD, EMA(20/50/200), ADX, ATR, Bollinger, StochRSI	chart_prompt_builder
market_context.py	Bitget + Bybit + Binance + OKX + alternative exchanges avg)	50 min	Funding rates, S/R, Fear & Greed, Open Interest ratio, Open Int	market_prompt_builder
nansen_client.py	Nansen.ai screener	30 min	On-chain wallet activity: netflow direction (accumulating/distributing), trader count	onchain_prompt_builder
grok_client.py	xAI Responses API	30 min	X/Twitter sentiment, recent news, social quality score change	news_prompt_builder
gemini_client.py	Google Generative Language API	30 min	Independent pre-proof score 1-10 with analysis (scale)	gemini_prompt_builder

5.1 Chart Context Architecture

The chart pipeline is split into three pure modules for testability and maintainability:

Module	Responsibility	Key functions
chart_indicators.py	Pure indicator computation — no API calls, no database	compute_composite_effects, compute_macd, compute_ema_alignment, compute_
chart_sr.py	S/R detection with ATR-relative tolerance	detect_support_resistance (ATR-relative clustering, exponential d
chart_context.py	Thin orchestrator: fetches candles, caches, gets indicators	get_candles (6 days, 10-min cache), compute_indicators (delegate

VMC Cipher A/B (WaveTrend Oscillator): implemented from scratch matching TradingView's VuManChu Cipher B defaults (n1=10, n2=21). Provides gold_buy, buy, and sell signals based on oversold/overbought crossovers. Adds a 7th confluence signal to the scoring system. Rendered as a separate oscillator pane below the candlestick chart in the browser.

6. Automation Agents

Automation agents run continuously in background threads. They require no user interaction — they watch the market, sync positions, and fire alerts automatically.

6.1 Scanner Scheduler — `scanner_scheduler.py`

Runs the full 3-stage scanner pipeline every 30 minutes. First scan fires 5 minutes after app startup (allowing exchange sync to complete). After every run that produces results above the score threshold:

1. Sends a Telegram HTML alert with symbol, direction, score, entry zone, SL, TP1, TP2, R:R, and urgency.
2. Saves each alerted setup to `analyzed_calls` with `analyst='scanner'`. This is critical for automatic position linking — when a scanner-alerted position opens on the exchange, `check-matches` auto-confirms the link without user action.
3. Deduplicates by (symbol, direction) within a 4-hour window to prevent spam on consecutive scans.

6.2 Exchange Sync — `bitget_sync.py` / `blofin_sync.py`

Runs every 5 minutes in a background thread. Uses cursor-based pagination to catch all closed positions regardless of holding duration. Key behaviours:

Feature	What it does
Auto-close calls	When a position closes, finds the linked <code>analyzed_call</code> and marks it closed. Records which TP or SL was
Market regime tagging	Tags each position bull/bear/range at entry time using BTC EMA50/200 cross (<code>get_btc_regime</code>). Enables
MFE/MAE tracking	Records Maximum Favourable Excursion and Maximum Adverse Excursion for each trade. Used for the
Deduplication	Idempotent — safe to run every 5 minutes. Uses exchange order IDs to prevent duplicate entries regardl
Catch-up window	On startup, fetches trades from the last 48 hours to recover any missed during downtime (Pi restart, netw

7. Prompt Architecture & Caching

How prompts are built and cached is one of the most important architectural decisions in the system — it directly affects both cost and accuracy.

7.1 The Stable / Dynamic Split

Anthropic's prompt caching works by storing a prefix that is byte-for-byte identical across calls. When the cached prefix is reused, Anthropic charges approximately 10% of the normal input token price for the cached portion. However, if any live data (funding rates, chart indicators, market context) is included in the cached block, the cache key changes every few minutes and cache hits never occur — you pay full price on every call.

The fix: **build_stable_prefix()** returns only content that changes at most weekly (rulebook + calibration + pattern strengths). **build_context()** returns dynamic content that changes per call (backtest insights, market data, chart indicators, Nansen, Grok, similar trades). The stable prefix gets `cache_control: ephemeral` — the dynamic context does not.

Block	Content	Cache?	Changes how often
Stable prefix (Block 1)	Rulebook (10 rules), calibration feedback, top-3 patterns, strengths	✓ YES cache_control: ephemeral	Weekly (when 5+ new trades)
Dynamic context (Block 2)	Backtest insights, live market context, chart indicators, Nansen, Grok, similar trades	✗ NO	Every call (market data, every 5 min)
Variable prompt (Block 3)	Call text, position sizing, CoT from previous same symbol analysis	✗ NO	Every call

7.2 Backtest Feedback Loop

Every Claude analysis prompt begins with a compact historical performance block injected by **get_backtest_context()** in `analytics.py`. This gives Claude specific, numerical context about YOUR trading patterns before it scores any new call:

Example backtest context injected into a prompt:

```
BACKTEST INSIGHTS: Recent form: 72% WR last 20 · streak WWLWW · avg +$8.40 Breakout setups: 100% WR (6 trades) avg +$7.00 BTCUSD Long: 75% WR (12 trades) avg +$12.50 ■ Wednesday: caution (57% WR, -$355 total P&L;) ■ 21:00 UTC: weak hour (70% WR, -$1831 total)
```

This is not generic advice — it is derived from the user's actual trade history in real time. Claude sees both the opportunity signal (technical setup) and the historical context (does this trader actually profit from this type of setup at this time of day?) before assigning a score. This is the primary mechanism for improving accuracy as trade history grows.

7.3 CoT Learning Loop

When Claude analyzes a call, its step-by-step reasoning (the 'thinking' field in the JSON response) is stored as `cot_reasoning` in the database. The next time the same symbol is analyzed, the previous reasoning is injected into the prompt as `PREVIOUS ANALYSIS` context. Claude can then explicitly compare: 'Last time I analyzed ARKMUSD, I flagged the SL as too close to the 4H noise floor. Has that changed?' This enables detection of repeated mistakes and continuous refinement without any retraining.

8. Position Auto-Linking System

A key feature is that scanner-alerted trades and manually analyzed calls are automatically linked to the corresponding live positions — without requiring the user to confirm each match manually.

8.1 How the link is established

Scenario	Auto-link behaviour
Scanner sends Telegram alert for NOTUSDT Long call .	scanner._persist_setups() saves the setup to analyzed_calls (analyst='scanner', status='').
User ran call analyzer for NOTUSDT Long, position closed, when the position closed.	When a new NOTUSDT Long opens, check-ma
Trade came from Telegram but not call live position analyzed (WSXUSDT call).	'Analyze First' button. Clicking it navigates to the call analyz
Scanner signal didn't save (scored below threshold) but position was opened.	check

8.2 What appears in Live Trades when linked

Once a position is linked to a call, the live trades card shows a **Call Targets Panel** with: distance from mark price to SL, TP1, TP2, and the call's average entry. A TP1-reached alert fires when mark price crosses TP1, with an automatic break-even stop suggestion. Positions with a linked call also show the setup score, trade type, and R:R ratio from the original analysis.

9. Accuracy Measurement & 85% Target

The accuracy target of $\geq 85\%$ means: when the consensus score is ≥ 6 and confidence is 'high' (both Claude and Gemini agree within 1 point), the trade should be profitable at least 85% of the time. This is measured by `scripts/backtest_consensus.py`.

9.1 Three hypotheses tested

Hypothesis	Claim	How measured
H1: Claude-only	Score ≥ 6 from Claude alone predicts a profitable trade	<code>outcome_trade_win()</code> for all calls with <code>setup_score</code> $\geq N$
H2: Consensus	Agreement between Claude and Gemini (within 1 point) for calls with consensus_score $\geq N$ AND confidence='high'	<code>outcome_trade_win()</code> for calls with <code>consensus_score</code> $\geq N$ AND <code>confidence</code> ='high'
H3: Divergence avoidance	Calls with $ \Delta > 2$ (REVIEW flag) have lower outcome_trade_win rate	<code>outcome_trade_win()</code> for calls with $ \text{claude_score} - \text{gemini_score} > 2$

Current status: 5 outcome-recorded calls (need ≥ 20 for statistical confidence). The system is now accumulating evidence — every new call saved stores `gemini_score` and `consensus_score`, and every outcome recorded improves the backtest context injected into future prompts. The 85% target becomes measurable after ~15-20 more outcome-recorded calls.

Run the backtest at any time: `python3 scripts/backtest_consensus.py --host <pi-ip>:8082`. Add `--live` to re-score the last 20 calls with Gemini live (uses API credits).

10. Complete Agent Reference

Agent / Module	Type	Model	Trigger	Token budget
call_analyzer	Analysis	Sonnet 4.6	On demand	~4,000 in / 4,096 out
scanner (batch)	Analysis	Sonnet 4.6	30 min auto	~5,500 in / 14,400 out
scanner (quick)	Analysis	Haiku 4.5	Per finalist	~1,200 in / 120 out
advisor	Analysis	Sonnet 4.6	On demand	~4,000 in / 4,096 out
rulebook	Analysis	Sonnet 4.6	Weekly / manual	~3,000 in / 2,048 out
hindsight	Analysis	Haiku 4.5	Batch on demand	~800 in / 512 out
live_trade	Analysis	Haiku 4.5	Per click	~600 in / 768 out
trade_grader	Analysis	Haiku 4.5	Per closed trade	~700 in / 350 out
limit_analyzer	Analysis	Sonnet 4.6	Per limit order	~2,000 in / 768 out
pattern_detector	Analysis	Sonnet 4.6	Via advisor	~2,500 in / 1,200 out
Gemini 2.0 Flash	External AI	Gemini 2.0	Parallel w/ call	~300 in / 200 out
xAI Grok 3 Fast	External AI	Grok 3	Parallel w/ call	~250 in / 130 out
Nansen screener	Data	—	Per scan / call	1 API credit per run
chart_context	Data	—	Per analysis	Bitget REST (cached)
market_context	Data	—	Per analysis	4 exchanges + 2 APIs
scanner_scheduler	Automation	—	Every 30 min	Spawns scanner + Telegram
bitget_sync	Automation	—	Every 5 min	Bitget REST cursor
blofin_sync	Automation	—	Every 5 min	Blofin REST cursor

Trading Journal v1.0.1 · Self-hosted on Raspberry Pi 5 · Built with Claude Code · github.com/anvilfilbert/Auto-Crypto-Tradingjournal